Guia do Aluno

Curso de Automação de Testes com Playwright + IA — Hat Store

Curso de Automação de Testes com Playwright + IA — Hat Store
- Três premissas do curso
- Estrutura do repositório
- Módulos
- Início rápido (depois do setup)

Módulo 00 — Setup do ambiente (Windows)
- 0.1 Contas necessárias (gratuitas)
- 0.2 Git e VS Code
- 0.3 Node.js (LTS)
- 0.4 Podman Desktop (para subir a Hat Store)
- 0.5 Clonar o curso e subir a Hat Store
- 0.6 Rodar o primeiro teste
- 0.7 IA gratuita: GitHub Copilot Free + plano B
- Problemas comuns

Módulo 01 — Teoria de Automação de Testes
- 1.1 O que é automação de testes
- 1.2 Pirâmide de testes (na Hat Store)
- 1.3 Tipos de teste que a automação cobre
- 1.4 Quando automatizar (e quando não)
- 1.5 Flakiness (testes instáveis)
- 1.6 Massa de dados e isolamento
- 1.7 Métricas de uma suíte saudável
- Exercício (sem código)

Módulo 02 — Fundamentos do Playwright
- 2.1 O que é o Playwright e como criar um projeto do zero
- 2.2 Tour pelo projeto do curso
- 2.3 Descobrindo locators **sem o código-fonte**: DevTools
- 2.4 `codegen` e "Pick locator"
- 2.5 Primeiro teste: ações e asserções
- 2.6 Rodando e depurando
- Exercício (dever de casa)

Módulo 03 — Padrões de projeto e boas práticas
- 3.1 Por que Page Object?

- 3.2 Refatorando a busca para `HomePage`
- 3.3 `LoginPage`: cadastro e login
- 3.4 Cenários positivos e negativos
- 3.5 Fixtures: injetando Page Objects nos testes
- 3.6 Exercício
- 3.7 Dica de IA: refatorar para Page Object

Módulo 04 — Automação frontend avançada
- 4.1 Organizando testes por contexto de negócio
- 4.2 Massa de dados: fixa x dinâmica
- 4.3 Fixtures de massa dinâmica
- 4.4 Cenários combinados
- 4.5 Regressão visual
- 4.6 e 4.7 Fluxo E2E completo
- 4.8 Scripts do `package.json`
- Exercício
- 4.9 Dica de IA: gerar massa de dados

Módulo 05 — Testes de API com Playwright
- 5.1 Swagger: a fonte da verdade (sem código-fonte)
- 5.2 A fixture `request` e o primeiro GET
- 5.3 Organização: `tests/front` x `tests/api` e a camada de API
- 5.4 Autenticação: fixture de API com JWT
- 5.5 Cobrindo os endpoints principais
- 5.6 Testes de contrato
- 5.7 Exercício
- 5.8 Dica de IA: gerar o schema a partir da resposta

Módulo 06 — IA aplicada a testes e MCP do Playwright
- 6.1 Onde a IA ajuda de verdade
- 6.2 Onde a IA erra (e por quê)
- 6.3 Prompt com **fatos da tela**, não do código
- 6.4 Mão na massa com o Copilot Free (VS Code)
- 6.5 O mesmo fluxo no chat web (ChatGPT/Gemini free)
- 6.6 Debug com IA
- 6.7 O que é MCP e como ligar o MCP do Playwright
- 6.8 MCP na prática: a IA explora a Hat Store e gera o teste
- 6.9 Playwright Test Agents (planner, generator, healer)
- 6.10 Checklist de revisão de código gerado por IA
- Segurança e IA
- 6.11 Se você tiver acesso ao código-fonte
- Exercício com IA

Módulo 07 — Fechamento e próximos passos
- 7.1 Resolução do exercício de IA
- 7.2 O que você construiu
- Próximos passos (para continuar estudando)

Curso de Automação de Testes com Playwright + IA — Hat Store

Curso introdutório de automação de testes **E2E (tela) e de API** com Playwright, com um módulo de **IA aplicada a testes**, incluindo MCP. Foi pensado para quem tem pouca ou nenhuma experiência com automação ou com IA.

Baseado no repositório de aulas start-aula-playwright.

Três premissas do curso

1. **Sem acesso ao código-fonte.** Você testa a Hat Store do mesmo jeito que um QA que só recebeu a URL do ambiente: pela tela, pelo DevTools, pelo `codegen`, pelo Swagger e com a IA olhando a página no navegador (MCP). O código da loja fica em `hatstore-app/` só para você conseguir subir a aplicação.
2. **Só ferramentas gratuitas.** Node, VS Code, Podman, Playwright e **GitHub Copilot Free**. Como plano B, ChatGPT ou Gemini na versão gratuita pelo navegador.
3. **Aulas curtas.** Cada vídeo tem até 10 minutos e trata de um assunto só.

Estrutura do repositório

```
├── 00-setup/                    instalação passo a passo (comece aqui)
├── 01-teoria-automacao-testes/
├── 02-fundamentos-playwright/
├── 03-padroes-boas-praticas/
├── 04-frontend-avancado/
├── 05-testes-api/
├── 06-ia-e-mcp/
├── 07-fechamento/
├── hatstore-app/                aplicação sob teste (sobe com Podman)
├── playwright-project/          projeto de automação (onde você programa)
└── docs/                        Guia do Aluno em PDF
```

Módulos

#	Módulo	O que você aprende
00	Setup	Instalar Git, Node, VS Code, Podman; subir a Hat Store; rodar o 1º teste; ativar Copilot Free
01	Teoria	Pirâmide, tipos de teste, quando automatizar, flakiness, massa de dados
02	Fundamentos do Playwright	Projeto do zero, locators sem código-fonte, `codegen`, assertions, UI mode, trace
03	Padrões e boas práticas	Page Object, login e cadastro, cenários negativos, fixtures
04	Frontend avançado	Massa de dados, factory, cenários combinados, regressão visual, E2E completo
05	Testes de API	Swagger, `request`, autenticação JWT, testes de contrato
06	IA e MCP	Prompts com fatos da tela, Copilot Free, chat web, MCP do Playwright, Test Agents
07	Fechamento	Resolução do exercício de IA e próximos passos

Início rápido (depois do setup)

```
cd hatstore-app
podman compose up --build -d
cd ..\playwright-project
npm install
npx playwright install chromium
npm test
```

Módulo 00 — Setup do ambiente (Windows)

Ao final deste módulo você terá:

- a **Hat Store rodando** em http://localhost:8080;
- o **projeto de testes** rodando e o relatório aberto;
- o **VS Code com Copilot Free** configurado (e um plano B de IA no navegador).

> Tempo estimado: 40–60 min (a maior parte é download). Você vai precisar de **permissão de administrador** para instalar o WSL/Podman. Se não tiver, peça ao suporte antes de começar.

0.1 Contas necessárias (gratuitas)

Conta	Para quê	Link
GitHub	Clonar o curso e usar o Copilot Free	https://github.com/signup

> **Nota**
>
> O repositório do curso é **privado**. Envie seu usuário do GitHub para os instrutores e aceite o convite que chega por e-mail antes de seguir para o passo 0.5.

0.2 Git e VS Code

Opção A — pelo terminal (PowerShell):

```
winget install --id Git.Git -e
winget install --id Microsoft.VisualStudioCode -e
```

Opção B — pelo site: baixe e instale com as opções padrão:

- Git: https://git-scm.com/download/win
- VS Code: https://code.visualstudio.com/

Feche e abra o PowerShell de novo e confira:

```
git --version
code --version
```

Extensões do VS Code (aba *Extensions*, `Ctrl+Shift+X`):

Extensão	ID
Playwright Test for VSCode	`ms-playwright.playwright`
GitHub Copilot Chat	`github.copilot-chat`

> Ao abrir a pasta do curso, o VS Code sugere essas extensões automaticamente (arquivo `.vscode/extensions.json`).

0.3 Node.js (LTS)

```
winget install --id OpenJS.NodeJS.LTS -e
```

Ou baixe o instalador **LTS** em https://nodejs.org/. Feche e abra o PowerShell e confira:

```
node --version   # v20 ou superior
npm --version
```

> Se o PowerShell reclamar de *execution policy* ao rodar `npm` / `npx`, execute uma vez: `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`

0.4 Podman Desktop (para subir a Hat Store)

O Podman roda containers, como o Docker, e é gratuito. A Hat Store usa dois containers: banco PostgreSQL + aplicação.

1. Instale:

   ```
   winget install --id RedHat.Podman-Desktop -e
   ```

   ou baixe em https://podman-desktop.io/downloads.
2. Abra o **Podman Desktop**. Na primeira execução ele mostra um assistente:
   - Se pedir para instalar o **WSL2**, aceite. Pode ser necessário **reiniciar o Windows**.
   - Clique em **Create new Podman machine** e aceite os valores padrão.
   - Em *Settings* → *Resources*, confirme que a máquina está **Running**.
3. Instale o **Compose**: no Podman Desktop, vá em *Settings* → *Resources* → *Compose* → *Setup* (ou no banner "Compose not installed"). Siga o assistente.
4. Confira no PowerShell:

```
podman --version
podman machine list        # a máquina deve aparecer como "Currently running"
podman compose version
```

> Se você reiniciar o computador, abra o Podman Desktop (ou rode `podman machine start`) antes de subir a Hat Store.

0.5 Clonar o curso e subir a Hat Store

```
cd $HOME\Documents
git clone https://github.com/javierdelamaza/curso-automacao-playwright-hatstore.git
cd curso-automacao-playwright-hatstore
code .
```

> Na primeira vez, o Git abre uma janela para você **entrar na sua conta do GitHub**.

No terminal do VS Code (`Ctrl+`):

```
cd hatstore-app
podman compose up --build -d
```

Espere terminar (na primeira vez leva alguns minutos). Depois abra **http://localhost:8080**.

Explore a loja como usuário (é o seu primeiro "teste exploratório"):

- busque "Panamá", filtre por categoria e por preço;
- crie uma conta, faça login, adicione um chapéu ao carrinho, aplique o cupom `HATOFF`;
- finalize uma compra com PIX e veja em **Meus Pedidos**;
- abra o Swagger: http://localhost:8080/swagger/index.html.

> **Dica**
>
> Anote o que você achou estranho. Testar não é só automatizar, e o curso vai mostrar que a Hat Store tem bugs de verdade.

0.6 Rodar o primeiro teste

Ainda no terminal do VS Code:

```
cd ..\playwright-project
npm install
npx playwright install chromium
npm test
```

O resultado esperado é algo como `43 passed` (alguns aparecem como *skipped*: são os exercícios). Abra o relatório:

```
npm run report
```

Também dá para rodar pela extensão: ícone de **tubo de ensaio** (Testing) na barra lateral → ▶ ao lado de um teste. Marque **Show browser** para ver o navegador.

0.7 IA gratuita: GitHub Copilot Free + plano B

Copilot Free (principal)

1. No VS Code, clique no ícone do **Copilot** na barra de status (canto inferior direito) → **Sign in** / **Use Copilot for free** → entre com sua conta do GitHub.
2. Abra o chat: `Ctrl+Alt+I`. No seletor de modo, você vai usar:
   - **Ask**: perguntas e explicações;
   - **Agent**: a IA edita arquivos e usa ferramentas, como o MCP do Playwright (módulo 06).
3. O plano Free tem **cota mensal limitada** de completions e de mensagens no chat. Confira os limites atuais em https://github.com/features/copilot/plans. Dica: use o modo **Ask** para dúvidas e guarde o **Agent** para as práticas do módulo 06.

> **Atenção**
>
> Verifique a política da sua empresa sobre o uso de contas pessoais de IA. No curso só usamos a Hat Store, que é fictícia. **Nunca** cole código, dados ou credenciais de projetos reais em uma IA de conta pessoal ou gratuita.

Plano B — chat no navegador (sem instalar nada)

Se a cota do Copilot acabar ou ele não estiver disponível: ChatGPT (https://chatgpt.com) ou Gemini (https://gemini.google.com) no plano gratuito. O fluxo é *copiar e colar*: você leva para o chat o HTML ou o erro e traz a resposta para o VS Code. O módulo 06 ensina esse fluxo também.

Problemas comuns

Sintoma	Causa provável	Solução
`Cannot connect to Podman`	Máquina Podman parada	`podman machine start` ou abra o Podman Desktop
`podman compose` não encontrado	Compose não instalado	Podman Desktop → Settings → Resources → Compose → Setup
`port 8080 already in use`	Outra aplicação na porta	Feche a aplicação ou pare containers antigos (`podman ps` , `podman stop <id>`)
Todos os testes falham com `ECONNREFUSED`	Hat Store não está no ar	Suba com `podman compose up -d` na pasta `hatstore-app`
`npx` bloqueado no PowerShell	Execution policy	`Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`
Testes falham por estoque/dados estranhos	Banco "sujo" de muitas execuções	`podman compose down -v` e `podman compose up --build -d`
Firefox: `Host system is missing dependencies` / `Failed to launch`	Antivírus ou política corporativa bloqueando o Firefox do Playwright	O curso usa só o Chromium (`npm test`). O Firefox é opcional; ignore ou peça liberação ao suporte

Próximo módulo: 01 — Teoria de Automação de Testes

Módulo 01 — Teoria de Automação de Testes

Antes de escrever código, precisamos saber **o que** vale a pena automatizar e **por quê**.

1.1 O que é automação de testes

É usar scripts para executar verificações que uma pessoa faria manualmente: clicar, preencher formulários, conferir textos na tela, validar respostas de API.

O objetivo **não** é substituir o teste humano. A automação serve para:

- repetir a regressão de forma rápida e sempre igual;
- rodar em vários navegadores e em paralelo;
- rodar no CI/CD e barrar uma entrega quebrada;
- liberar o QA para o teste exploratório, que é onde se acham os bugs mais interessantes.

1.2 Pirâmide de testes (na Hat Store)

```
      /\        E2E / UI: poucos, lentos, cobrem fluxos críticos
     /  \
    /----\      API / Integração: rápidos, cobrem regras e contratos
   /      \
  /--------\    Unitários: muitos, rapidíssimos (responsabilidade do time de dev)
```

Camada	Quem costuma fazer	Na Hat Store
Unitário	Dev (precisa do código-fonte)	Já existe no projeto (Go e Jest). **Não é foco do curso.**
API	QA e Dev	`GET /api/hats`, `POST /auth/login`, `POST /api/pedido` … pelo **Swagger**
E2E / UI	QA	Buscar, filtrar, cadastrar, logar, comprar com PIX/Boleto

> O QA consegue atuar nas camadas de **API e E2E sem ler o código-fonte**. Basta o ambiente no ar, o Swagger e a tela. É o cenário deste curso.

1.3 Tipos de teste que a automação cobre

- **Funcional:** o sistema faz o que deveria. Ex.: o cupom `HATOFF` dá 20% de desconto no carrinho.
- **Regressão:** o que funcionava continua funcionando depois de uma mudança.
- **Contrato (API):** a resposta mantém o formato combinado (campos, tipos).

- **Visual:** a tela não "quebrou" o layout (comparação de screenshots).
- **Não funcionais** (fora do escopo, mas bom conhecer): performance, carga, segurança, acessibilidade.

1.4 Quando automatizar (e quando não)

Automatize quando o cenário é:

- **repetitivo:** roda em toda regressão;
- **estável:** a tela ou o fluxo não muda toda semana;
- **de alto risco:** login, checkout, pagamento.

Deixe para depois, ou mantenha manual:

- telas em redesenho constante;
- casos raríssimos ou que exigem julgamento humano ("o botão parece desalinhado");
- testes exploratórios: eles *descobrem* o que vale automatizar.

1.5 Flakiness (testes instáveis)

Um teste é *flaky* quando passa e falha sem que nada mude. Causas comuns:

Causa	Exemplo	Como evitar
Espera fixa	`waitForTimeout(3000)`	Esperar uma **condição** (`expect(...).toBeVisible()`)
Dependência entre testes	Teste B usa usuário criado no teste A	Cada teste cria os próprios dados
Seletor frágil	`div > div:nth-child(3)`	Locators por papel, texto, label ou `data-testid`
Ambiente	Container ainda subindo	Subir o ambiente antes e conferir que está saudável

O Playwright resolve boa parte disso com **auto-waiting** (módulo 02). O resto é disciplina de quem escreve o teste.

1.6 Massa de dados e isolamento

- **Cada teste roda sozinho**, em qualquer ordem.
- **Massa fixa:** vem do banco inicial e não muda (catálogo de 15 chapéus, cupom `HATOFF`).
- **Massa dinâmica:** é criada a cada teste (usuário novo com e-mail e CPF únicos).
- Na Hat Store isso importa. Ela **não deixa** repetir e-mail nem CPF, bloqueia o login depois de 5 tentativas erradas e o cupom só vale uma vez por CPF.

1.7 Métricas de uma suíte saudável

- **Cobertura de fluxos críticos:** login, busca e compra têm teste?
- **Tempo de execução:** dá para rodar a cada Pull Request?
- **Taxa de flakiness:** quantas vezes é preciso "rodar de novo para passar"?
- **Manutenção:** se um botão muda de lugar, quantos arquivos você precisa alterar? (O Page Object, no módulo 03, existe para diminuir esse número.)

Exercício (sem código)

Com a Hat Store aberta no navegador, escreva:

1. Os **3 fluxos** que você automatizaria primeiro e por quê.
2. **1 fluxo** que você *não* automatizaria agora, com a justificativa.
3. **2 dados** que precisam ser únicos a cada execução de teste.
4. **Bônus:** algo que parece bug. (Dica: compare o que a documentação em `/documentacao.html` diz com o que a tela faz.)

Próximo módulo: 02 — Fundamentos do Playwright

Módulo 02 — Fundamentos do Playwright

2.1 O que é o Playwright e como criar um projeto do zero

O Playwright é um framework open source da Microsoft para automatizar navegadores (Chromium, Firefox e WebKit) com uma única API. Tem versões em JavaScript/TypeScript, Python, Java e .NET. Neste curso usamos **JavaScript**.

O que ele tem de diferente:

- **Auto-waiting:** antes de clicar ou preencher, espera o elemento existir, ficar visível, habilitado e estável.
- **Web-first assertions:** `expect(locator).toBeVisible()` tenta de novo até o timeout, sem `sleep`.
- **Isolamento:** cada teste roda num contexto de navegador novo (cookies e localStorage limpos).
- **Ferramentas de debug:** UI mode, trace viewer, vídeo e screenshot na falha, `codegen`.

Criando um projeto do zero (para aprender; o projeto do curso já vem pronto):

```
mkdir meu-projeto-playwright
cd meu-projeto-playwright
npm init playwright@latest
```

Respostas no assistente:

Pergunta	Resposta
TypeScript or JavaScript?	**JavaScript**
Where to put your end-to-end tests?	**tests**
Add a GitHub Actions workflow?	**N**
Install Playwright browsers?	**Y**

2.2 Tour pelo projeto do curso

```
playwright-project/
├── playwright.config.js   configuração (baseURL, navegadores, trace, relatório)
├── package.json           dependências e scripts (npm test, npm run test:ui...)
├── pages/                 Page Objects (módulo 03)
├── fixtures/              fixtures customizadas (módulos 03 e 04)
├── data/                  massa de dados fixa (módulo 04)
├── factories/             massa de dados dinâmica (módulo 04)
├── api/                   cliente da API e contratos (módulo 05)
├── tests/
│   ├── front/             testes de tela
│   ├── api/               testes de API
│   └── exercicios/        exercícios (test.fixme)
└── solucoes/              soluções dos exercícios
```

Pontos importantes do `playwright.config.js`:

- `baseURL: 'http://localhost:8080'` → no teste basta escrever `page.goto('/')`.
- `trace: 'on-first-retry'`, `screenshot: 'only-on-failure'`, `video: 'retain-on-failure'` → evidências quando falha.
- `projects` → `chromium` (padrão), `firefox`, `visual` (módulo 04) e `solucoes`.

2.3 Descobrindo locators sem o código-fonte: DevTools

Você não precisa do código da aplicação para saber como localizar um elemento. O navegador mostra tudo.

1. Abra http://localhost:8080 no Chrome ou no Edge.
2. Clique com o botão direito no campo de busca → **Inspecionar** (ou `F12`).
3. Você vê o HTML real:

   ```html
   <input type="text" id="search-hat" placeholder="Buscar chapéu...">
   ```

4. Aba **Accessibility**, no painel do DevTools: mostra o **papel** (role) e o **nome acessível** do elemento. É isso que o `getByRole` usa.

Ordem de preferência de locators (recomendação oficial do Playwright):

Prioridade	Locator	Exemplo na Hat Store
1	`getByRole`	`page.getByRole('button', { name: 'Adicionar ao carrinho' })`
2	`getByLabel`	`page.getByLabel('Cupom de desconto:')`
3	`getByPlaceholder` / `getByText`	`page.getByPlaceholder('Buscar chapéu...')`
4	`getByTestId`	`page.getByTestId('...')`, quando o dev adiciona `data-testid`
5	CSS / id	`page.locator('#login-email')`, `page.locator('.hat-card')`

Evite: `div > div:nth-child(3) > span` e XPath longos. Quebram com qualquer mudança de layout.

Quando não existe um locator estável: peça um `data-testid` ao dev

Os **cards de produto** da Hat Store não têm role, label nem id. Só a classe CSS `.hat-card`. Se o time de front mudar essa classe por causa de estilo, todos os testes quebram.

Você descobre isso **só olhando o DevTools**, sem ler código. A ação de QA é abrir um pedido para o time de desenvolvimento:

> **Título:** Adicionar `data-testid` nos cards de produto da home **Onde:** Home → lista de chapéus (cada card) **Pedido:** `data-testid="hat-card"` no card e `data-testid="hat-card-price"` no preço **Motivo:** hoje o único seletor disponível é a classe CSS `.hat-card`, que pode mudar por estilo e quebrar a automação.

Isso é **testabilidade**, e é trabalho do QA também.

2.4 `codegen` e "Pick locator"

O Playwright grava o que você faz e gera o código com locators **reais**:

```
npm run codegen      # = npx playwright codegen http://localhost:8080
```

- Abre um navegador e a janela do **Inspector**.
- Clique e digite na loja → o código aparece no Inspector.
- Botão **Pick locator**: passe o mouse sobre um elemento e veja o melhor locator.

Na extensão do VS Code (aba Testing): **Record new** e **Pick locator** fazem o mesmo, direto no editor.

> O `codegen` é um **ponto de partida**. Revise: ele não cria asserções boas e às vezes escolhe um locator pior do que você escolheria.

2.5 Primeiro teste: ações e asserções

Arquivo tests/front/01-primeiro-teste.spec.js:

```
import { test, expect } from '@playwright/test';

test('busca por "Panamá" retorna exatamente um chapéu', async ({ page }) => {
  await page.goto('/');

  await page.getByPlaceholder('Buscar chapéu...').fill('Panamá');

  await expect(page.locator('.hat-card')).toHaveCount(1);
  await expect(page.locator('.hat-card h3')).toHaveText('Chapéu Panamá');
});
```

Estrutura de todo teste: **preparar** → **agir** → **verificar** (Arrange, Act, Assert).

Ações mais usadas	Asserções mais usadas
`goto`, `click`, `fill`, `check`, `selectOption`, `press`, `pressSequentially`	`toBeVisible`, `toHaveText`, `toContainText`, `toHaveCount`, `toHaveURL`, `toHaveValue`, `toBeEnabled`

2.6 Rodando e depurando

```
npm test                  # todos os testes, sem abrir navegador (headless)
npm run test:headed       # vendo o navegador
npm run test:ui           # UI mode: linha do tempo, DOM de cada passo (recomendado para
        aprender)
npx playwright test tests/front/busca.spec.js      # um arquivo
npx playwright test -g "Panamá"                    # testes cujo nome contém "Panamá"
npm run report            # relatório HTML da última execução
```

Quando um teste falha:

1. Leia a mensagem: *Expected* x *Received* e a linha marcada com `>`.
2. Abra o relatório → screenshot, vídeo e **trace** (linha do tempo com o DOM de cada passo).
3. Rode com `--debug` para executar passo a passo.

Exercício (dever de casa)

Em tests/exercicios/exercicios.spec.js, bloco **"Bloco 2 - filtros da home"**:

1. Remova o `.fixme` do teste de categoria **Crescer** e implemente.
2. Remova o `.fixme` do teste de **faixa de preço 100 a 130** e implemente.
3. Rode com `npm run test:exercicios`.

Descubra os locators pelo DevTools ou pelo `codegen`. A solução está em `solucoes/exercicios.spec.js`; só olhe depois de tentar.

Próximo módulo: 03 — Padrões e boas práticas

Módulo 03 — Padrões de projeto e boas práticas

3.1 Por que Page Object?

Sem padrão, cada teste repete os seletores. Se o placeholder da busca mudar, você altera 20 arquivos. O **Page Object Model (POM)** encapsula uma tela numa classe:

- **locators** ficam num só lugar;
- **ações** ganham nomes de negócio (`search`, `login`, `adicionarAoCarrinho`);
- o teste fica legível: descreve *o que* o usuário faz, não *como* a tela é montada.

3.2 Refatorando a busca para `HomePage`

pages/HomePage.js (trecho):

```
export class HomePage {
  constructor(page) {
    this.page = page;
    this.searchInput = page.getByPlaceholder('Buscar chapéu...');
    this.hatCards = page.locator('.hat-card');
    this.hatCardTitles = page.locator('.hat-card h3');
  }

  async goto() {
    await this.page.goto('/');
    await this.hatCards.first().waitFor();
  }

  async search(term) {
    await this.searchInput.fill(term);
  }
}
```

Compare o primeiro teste com a versão refatorada em busca.spec.js.

Regras práticas:

- O Page Object **não faz asserções** (`expect`). Quem verifica é o teste.
- Métodos devolvem o resultado ou nada. Não escondem regras de negócio.
- Um Page Object por tela ou componente relevante: `HomePage`, `LoginPage`, `CartPage`, `CheckoutPage` e `PedidosPage`.

3.3 `LoginPage`: cadastro e login

pages/LoginPage.js cobre os dois formulários da tela `/auth.html`. Um detalhe que só aparece testando: o campo CPF tem **máscara**. Por isso usamos `pressSequentially` (digita tecla a tecla, como o usuário) em vez de `fill`.

```
await this.registerCpfInput.pressSequentially(cpf.replace(/\D/g, ''));
```

3.4 Cenários positivos e negativos

Um bom conjunto de testes cobre o **caminho feliz** e as **regras de rejeição**. Na Hat Store, pela tela e pela documentação em `/documentacao.html`, descobrimos:

Regra	Cenário negativo
Idade mínima 18 anos	Cadastro de menor é recusado
E-mail único	Cadastro com e-mail já usado é recusado
Senha forte (8+, maiúscula, minúscula, número, especial)	Senha fraca é recusada (exercício)
CPF no formato 000.000.000-00	CPF incompleto mostra erro (exercício)
Login	Senha errada ou usuário inexistente → "Erro no login: Não autorizado"

Veja login.spec.js e cadastro.spec.js.

> Repare que a mensagem de login inválido é **genérica**: não diz se o erro foi no e-mail ou na senha. Isso é uma boa prática de segurança, e o teste confirma que continua assim.

3.5 Fixtures: injetando Page Objects nos testes

Em vez de `const home = new HomePage(page)` em todo teste, criamos **fixtures** (fixtures/index.js):

```
import { test as base } from '@playwright/test';
import { HomePage } from '../pages/HomePage.js';

export const test = base.extend({
  homePage: async ({ page }, use) => use(new HomePage(page)),
});
```

E no teste:

```
import { test, expect } from '../../fixtures/index.js';

test('busca por produto existente retorna só ele', async ({ homePage }) => {
  await homePage.goto();
  await homePage.search('Panamá');
  await expect(homePage.hatCards).toHaveCount(1);
});
```

O Playwright só cria a fixture se o teste **pedir** por ela no parâmetro.

3.6 Exercício

Em tests/exercicios/exercicios.spec.js, bloco **"Bloco 3"**:

1. **Senha fraca** (`Senha1234` , sem caractere especial) deve ser recusada.
2. **CPF incompleto** deve mostrar erro no campo. Descubra no DevTools onde a mensagem aparece.

3.7 Dica de IA: refatorar para Page Object

Cole no Copilot Chat (modo **Ask**) um teste "cru" e o HTML do elemento copiado do DevTools (botão direito → *Copy* → *Copy outerHTML*):

> "Tenho este teste Playwright em JavaScript [cole o teste]. O HTML real da tela é [cole o HTML]. Crie um Page Object `CartPage` com os locators e métodos usados, priorizando `getByRole` / `getByLabel` . Não invente seletores que não estejam no HTML que colei."

Depois, **revise**: os locators existem? Os nomes dos métodos fazem sentido para o negócio?

Próximo módulo: 04 — Frontend avançado

Módulo 04 — Automação frontend avançada

4.1 Organizando testes por contexto de negócio

```
tests/front/
├── 01-primeiro-teste.spec.js   (histórico do módulo 02)
├── busca.spec.js               catálogo e busca
├── combinados.spec.js          filtros + busca juntos
├── login.spec.js
├── cadastro.spec.js
├── carrinho.spec.js
├── compra-e2e.spec.js          fluxo completo (@e2e)
└── visual.spec.js              regressão visual (@visual)
```

Um arquivo por funcionalidade, nomes de teste em linguagem de negócio e `test.describe` para agrupar. **Tags** no nome (`@e2e`, `@visual`) permitem rodar subconjuntos:

```
npm run test:e2e        # só @e2e
npm run test:front      # só a pasta front
```

4.2 Massa de dados: fixa x dinâmica

Fixa: data/catalogo.js reúne o que vem do banco inicial e não muda: produtos, cupom, endereço padrão, frete do Sudeste. Foi levantada **pela tela e pelo Swagger**, sem abrir o código.

```
export const PRODUTOS = {
  panama: { id: 1, nome: 'Chapéu Panamá', price: 120.0, categoria: 'importado' },
  // ...
};
```

Dinâmica: factories/usuarioFactory.js gera um usuário novo por teste: e-mail único, **CPF válido** (com dígitos verificadores) e data de nascimento de 25 anos.

```
const usuario = novoUsuario();                          // padrão válido
const menor = usuarioMenorDeIdade();                    // variação pronta
const senhaFraca = novoUsuario({ password: '123' });    // sobrescreve só o que interessa
```

Por que dinâmica? A Hat Store **não aceita** e-mail nem CPF repetidos, e o cupom só vale uma vez por CPF. Com dados fixos, o 2º run já falharia.

4.3 Fixtures de massa dinâmica

Em fixtures/index.js:

Fixture	O que entrega	Como
`usuarioCadastrado`	Um usuário novo, já cadastrado	`POST /auth/register` (API é mais rápida que a tela)
`token`	JWT desse usuário	`POST /auth/login`
`paginaLogada`	`page` já logada	Grava o token no `localStorage` antes de abrir o site

Duas formas de começar logado:

```
// 1) Pela tela (testa o login, mas é mais lento)
test.beforeEach(async ({ page, loginPage, usuarioCadastrado }) => {
  await loginPage.goto();
  await loginPage.login(usuarioCadastrado.email, usuarioCadastrado.password);
  await page.waitForURL('**/index.html');
});

// 2) Injetando o token (rápido, para testes que NÃO são sobre login)
test('...', async ({ paginaLogada, homePage }) => { /* já está logado */ });
```

Como descobrimos a chave `jwt_token`? No DevTools: **Application** → **Local Storage**, depois de fazer login pela tela. Sem código-fonte.

4.4 Cenários combinados

combinados.spec.js testa a **interação entre componentes**: categoria + faixa de preço, duas categorias juntas, filtro + busca. É nas combinações que aparecem os bugs que os testes isolados não pegam.

4.5 Regressão visual

```
test('home com catálogo completo', async ({ page, homePage }) => {
  await homePage.goto();
  await page.waitForLoadState('networkidle');
  await expect(page).toHaveScreenshot('home.png', { fullPage: true });
});
```

1. **Primeira execução:** gere a imagem de referência (*baseline*):

   ```
   npm run test:visual:update
   ```

2. **Próximas execuções:** compara pixel a pixel com a baseline:

   ```
   npm run test:visual
   ```

3. Se falhar, o relatório mostra **Expected / Actual / Diff** lado a lado.

Cuidados: a baseline depende do sistema operacional e das fontes (por isso cada aluno gera a sua e ela não vai para o Git neste curso). Dados que mudam, como datas e estoque, precisam de `mask`. A tolerância está configurada em `playwright.config.js` (`maxDiffPixelRatio`).

4.6 e 4.7 Fluxo E2E completo

compra-e2e.spec.js faz o caminho do cliente de ponta a ponta, dividido em `test.step` (cada passo aparece no relatório):

1. login no `beforeEach` (pela tela);
2. buscar "Gustavo" e adicionar ao carrinho;
3. checkout mostra nome e CPF do usuário logado;
4. preencher endereço → frete calculado ao sair do campo CEP (Sudeste, R$ 12.50);
5. pagar com PIX → modal "Pagamento PIX" com o código;
6. **Meus Pedidos** mostra o pedido com o produto e "PIX".

> **E2E é caro:** tenha poucos, cobrindo os fluxos que dão dinheiro ou dão problema. As variações de regra ficam nos testes de API (módulo 05).

4.8 Scripts do `package.json`

Script	O que faz
`npm test`	Toda a suíte no Chromium (sem visual)
`npm run test:front` / `test:api`	Só tela / só API
`npm run test:e2e`	Só os fluxos `@e2e`
`npm run test:visual` / `test:visual:update`	Comparar / gerar baseline visual
`npm run test:all-browsers`	Chromium + Firefox (instale com `npx playwright install firefox`)
`npm run test:exercicios` / `test:solucoes`	Exercícios / soluções

Exercício

Bloco **"Bloco 4"** em exercicios.spec.js: compra de um cliente do **Nordeste** (CEP `40010-000`) pagando com **Boleto**, começando já logado com a fixture `paginaLogada`. Valide região, valor do frete, o modal de pagamento e o pedido em Meus Pedidos.

4.9 Dica de IA: gerar massa de dados

Peça à IA:

> "Crie uma função JavaScript que gere um CPF **válido** (com dígitos verificadores) no formato 000.000.000-00, e explique o algoritmo em 5 linhas."

Depois **teste a resposta**: gere 5 CPFs e valide num validador online. A IA erra algoritmos com frequência. Compare com factories/usuarioFactory.js.

Próximo módulo: 05 — Testes de API

Módulo 05 — Testes de API com Playwright

5.1 Swagger: a fonte da verdade (sem código-fonte)

Abra http://localhost:8080/swagger/index.html. Ali estão todos os endpoints, os parâmetros, os corpos de requisição e de resposta e os códigos de status. Use **Try it out** para chamar a API pelo navegador.

Endpoint	Auth?	Para quê
`GET /api/hats`	não	Catálogo (filtros `categoria`, `min`, `max`)
`POST /auth/register`	não	Cadastro
`POST /auth/login`	não	Login → devolve `token` (JWT)
`GET /api/frete?cep=&quantidade=`	não	Cálculo de frete por região
`GET /api/me`	**sim**	Perfil do usuário logado
`POST /api/pedido`	**sim**	Criar pedido
`GET /api/meus-pedidos`	**sim**	Pedidos do usuário logado
`GET /api/pedidos?cpf=`	**sim**	Pedidos por CPF

> Outra fonte sem código: DevTools → aba **Network**. Faça login pela tela e veja a requisição `login`, com URL, corpo enviado e resposta. É assim que se descobre uma API que não tem Swagger.

5.2 A fixture `request` e o primeiro GET

O Playwright tem um cliente HTTP embutido. Não precisa de navegador, então roda muito rápido.

```
import { test, expect } from '@playwright/test';

test('GET /api/hats retorna o catálogo', async ({ request }) => {
  const response = await request.get('/api/hats');
  expect(response.status()).toBe(200);
  const chapeus = await response.json();
  expect(chapeus).toHaveLength(15);
});
```

5.3 Organização: `tests/front` x `tests/api` e a camada de API

Assim como o Page Object encapsula telas, a classe api/HatStoreApi.js encapsula endpoints:

```
export class HatStoreApi {
  constructor(request) { this.request = request; }
  listarChapeus(params) { return this.request.get('/api/hats', { params }); }
  login(email, password) { return this.request.post('/auth/login', { data: { email, password }
        }); }
  // ...
}
```

O teste fica: `await api.listarChapeus({ categoria: 'crescer' })`.

5.4 Autenticação: fixture de API com JWT

Endpoints protegidos exigem o cabeçalho `Authorization: Bearer <token>`. A fixture `apiAutenticada` (fixtures/index.js) cria um cliente HTTP que envia esse cabeçalho em toda chamada:

```
apiAutenticada: async ({ playwright, baseURL, token }, use) => {
  const contexto = await playwright.request.newContext({
    baseURL,
    extraHTTPHeaders: { Authorization: `Bearer ${token}` },
  });
  await use(new HatStoreApi(contexto));
  await contexto.dispose();
},
```

A cadeia de fixtures resolve tudo sozinha: `apiAutenticada` → `token` → `usuarioCadastrado`.

5.5 Cobrindo os endpoints principais

Arquivo	O que cobre
hats.spec.js	Catálogo, filtros, produto sem estoque
auth.spec.js	Cadastro (201, 409, 400), login (200, 401), `/api/me` com e sem token
frete.spec.js	Região por CEP (teste parametrizado), frete progressivo, CEP inválido
pedidos.spec.js	Criar pedido, sem token, meus pedidos, por CPF, cupom, pedido vazio

Teste parametrizado (um teste por linha da tabela):

```
const casos = [
  { cep: '01310-100', regiao: 'Sudeste', valor: 12.5 },
  { cep: '90010-000', regiao: 'Sul', valor: 15.0 },
];
for (const caso of casos) {
  test(`CEP ${caso.cep} é da região ${caso.regiao}`, async ({ api }) => { /* ... */ });
}
```

Bug encontrado pelos testes

O teste "*cupom HATOFF só pode ser usado uma vez por CPF*" revelou que **qualquer pedido com o cupom HATOFF devolve 500** ("Erro ao buscar chapéu"). O teste fica marcado assim:

```
test.fail(true, 'BUG: POST /api/pedido com cupom HATOFF devolve 500');
```

`test.fail` diz "espero que este teste falhe". A suíte continua verde **e o bug fica documentado**. Quando o dev corrigir, o teste passa a "falhar" e avisa para remover a marcação. Nunca apague um teste só porque ele encontrou um bug.

Outra divergência que a exploração revela, sem abrir código: a documentação (`/documentacao.html`) diz **25%** de desconto num lugar e **20%** em outro. A tela aplica 20%. Pergunta de QA: qual é a regra certa? Isso vira uma dúvida para o PO, não um "chute" no teste.

5.6 Testes de contrato

O teste de contrato não confere **valores**, confere o **formato**: campos obrigatórios, tipos e formatos. Se o backend renomear `price` para `preco`, o teste quebra **antes** do front.

Usamos **JSON Schema** (api/schemas.js) e a biblioteca gratuita **Ajv** (api/validarContrato.js):

```
export const chapeuSchema = {
  type: 'object',
  required: ['id', 'nome', 'price', 'quantidade', 'categoria', 'temEstoque'],
  properties: {
    id: { type: 'integer' },
    price: { type: 'number', minimum: 0 },
    categoria: { enum: ['nacional', 'importado', 'crescer'] },
    // ...
  },
};

expect(validarContrato(listaDeChapeusSchema, corpo)).toEqual([]);
```

Quando falha, a mensagem lista exatamente o que quebrou (ex.: `/0/price must be number`).

5.7 Exercício

Bloco **"Bloco 5"** em exercicios.spec.js: crie um pedido via API e valide que **cada pedido** de `GET /api/meus-pedidos` respeita o `pedidoSchema`.

5.8 Dica de IA: gerar o schema a partir da resposta

No Swagger, execute `GET /api/frete` e copie o JSON de resposta. Peça à IA:

> "Gere um JSON Schema (draft-07) para esta resposta [cole o JSON]. Marque todos os campos como obrigatórios, use `enum` para `regiao` com os valores Sudeste, Sul, Nordeste, Centro-Oeste e Norte."

Revise: tipos corretos? `integer` x `number`? Compare com o `freteSchema` do projeto.

Próximo módulo: 06 — IA e MCP

Módulo 06 — IA aplicada a testes e MCP do Playwright

A IA (Copilot, ChatGPT, Gemini...) acelera muito a escrita de testes. Mas **quem responde pela qualidade da suíte é você**, que revisa e roda o teste. Aqui a IA é copiloto, não piloto automático.

Tudo neste módulo funciona com **contas gratuitas** e **sem acesso ao código-fonte** da aplicação.

6.1 Onde a IA ajuda de verdade

- Gerar o esqueleto de um teste ou Page Object a partir de uma descrição **com fatos reais**.
- Sugerir locators melhores quando você mostra o HTML.
- Explicar uma mensagem de erro, um stack trace ou um timeout.
- Refatorar testes repetidos em fixtures e helpers.
- Gerar massa de dados (CPF válido, e-mails únicos) e JSON Schema.
- Transformar um cenário em linguagem natural ou Gherkin em código.

6.2 Onde a IA erra (e por quê)

Erro típico	Exemplo	Por que acontece
Locator inventado	`page.locator('#btn-login')` que não existe	A IA **não vê a página**, a menos que você mostre (HTML, `codegen`) ou dê um navegador a ela (MCP)
Espera fixa	`await page.waitForTimeout(3000)`	Padrão antigo, muito comum nos dados de treino
Teste genérico	`#username`, `#password`	Ela "completa" com o que é comum, não com o seu app
Asserção fraca	`expect(msg).not.toBeEmpty()`	Passa até com mensagem de erro
"Fazer passar"	Mockar a API, `test.skip`, afrouxar a asserção	Ela otimiza para "ficar verde", não para achar bug

6.3 Prompt com fatos da tela, não do código

Um bom prompt dá à IA **fatos verificáveis** tirados da aplicação rodando. Não é preciso ter o código-fonte.

Prompt fraco:

> Cria um teste de login pro meu site.

Prompt forte (modelo):

```
Contexto: projeto Playwright em JavaScript, testes em tests/front, usando as fixtures de
fixtures/index.js (homePage, cartPage...) e Page Objects em pages/.

Fatos da tela (copiados do DevTools/codegen):
- Campo de cupom: <input id="cupom" placeholder="Digite o cupom"> com <label>Cupom de
desconto:</label>
- Botão: <button id="aplicar-cupom">Aplicar</button>
- Após aplicar "HATOFF", aparece "Cupom aplicado!" e a linha
  "Cupom HATOFF aplicado: -R$ 24.00" para um item de R$ 120.00.

Tarefa: gere um teste que adiciona o "Chapéu Panamá", aplica HATOFF e valida a mensagem,
o desconto e o total.

Restrições: use os métodos existentes do CartPage; sem waitForTimeout; asserções com
toHaveText (valor exato); não invente seletores fora dos que listei.
```

Os quatro blocos são sempre os mesmos: **contexto** → **fatos** → **tarefa** → **restrições**.

De onde tirar os fatos (sem código-fonte):

- DevTools → botão direito no elemento → *Copy* → *Copy outerHTML*
- saída do `npm run codegen`
- `error-context.md` que o Playwright gera quando um teste falha (traz o *snapshot de acessibilidade* da página)
- Swagger e aba Network, para API

6.4 Mão na massa com o Copilot Free (VS Code)

1. Abra o chat (`Ctrl+Alt+I`) no modo **Ask**.
2. Anexe contexto: digite `#` e escolha arquivos (ex.: `#file:CartPage.js`, `#file:fixtures/index.js`). A IA passa a usar o **seu** padrão.
3. Cole o prompt forte do item 6.3 (troque pela tarefa que quiser).
4. Copie o código para um arquivo em `tests/front/`, rode (`npm test` ou ▶ na aba Testing) e **revise com o checklist (6.10)**.

> Economize a cota do plano Free: um prompt bem feito resolve em 1 mensagem o que 5 prompts vagos não resolvem.

6.5 O mesmo fluxo no chat web (ChatGPT/Gemini free)

Sem Copilot? O fluxo é o mesmo, só que por copiar e colar:

1. Abra o arquivo do Page Object e das fixtures e **cole o conteúdo** no chat, junto com o prompt.
2. Copie a resposta para o VS Code.
3. Rode, revise e, se falhar, cole o **erro** de volta no chat (item 6.6).

A diferença é só a comodidade. As boas práticas são as mesmas.

6.6 Debug com IA

Quando um teste falha, **não descreva o erro de memória**. Mande o erro real:

- a mensagem do terminal (Expected x Received, linha com `>`);
- o arquivo `test-results/<teste>/error-context.md`, que traz o estado da página no momento da falha;
- no relatório HTML e no UI mode existe o botão **Copy prompt**, que monta esse pacote pronto para colar na IA.

Prompt:

> "Este teste Playwright falhou. Erro e contexto abaixo. Explique a causa provável e proponha a correção **sem** usar waitForTimeout e **sem** afrouxar a asserção. [cole]"

6.7 O que é MCP e como ligar o MCP do Playwright

MCP (Model Context Protocol) é um padrão aberto que dá **ferramentas** a um agente de IA. O **Playwright MCP** dá à IA um **navegador de verdade**: ela abre a página, clica, digita e lê o *snapshot de acessibilidade* (o mesmo que o `getByRole` usa).

Resultado: acabam os locators inventados. A IA vê a página como ela é, **sem código-fonte**.

O repositório já traz a configuração em .vscode/mcp.json:

```
{
  "servers": {
    "playwright": { "type": "stdio", "command": "npx", "args": ["@playwright/mcp@latest"] }
  }
}
```

Para ativar:

1. Abra `.vscode/mcp.json` no VS Code e clique em **Start**, acima de `"playwright"`. Ou use `Ctrl+Shift+P` → **MCP: List Servers** → playwright → Start.
2. No chat, mude para o modo **Agent**.

3. Clique no ícone de **ferramentas** do chat e confira se as ferramentas `browser_*` do Playwright estão marcadas.

6.8 MCP na prática: a IA explora a Hat Store e gera o teste

Com a Hat Store no ar e o chat em modo **Agent**:

```
Use o Playwright MCP para abrir http://localhost:8080.
1. Busque por "Fedora", adicione ao carrinho e aumente a quantidade para 2.
2. Anote os locators que você usou (prefira role/label/placeholder).
3. Depois gere um teste em tests/front/ia-carrinho.spec.js que reproduz o fluxo e valida
   o total de R$ 300.00, usando as fixtures de #file:fixtures/index.js e os métodos de
   #file:pages/CartPage.js e #file:pages/HomePage.js. Não use waitForTimeout.
```

Acompanhe: o navegador abre e a IA "narra" o que viu. **Aprove cada ação** quando o VS Code pedir. Depois, rode o teste gerado e revise com o checklist.

> Cada passo do agente consome requisições da sua cota Free. Peça tarefas pequenas e bem definidas.

6.9 Playwright Test Agents (planner, generator, healer)

O Playwright traz três agentes prontos, que usam o MCP por baixo:

Agente	Faz o quê
planner	Explora a aplicação e escreve um **plano de testes** em Markdown
generator	Transforma o plano em arquivos `.spec.js` e verifica os locators ao vivo
healer	Roda um teste que falhou, investiga a página e propõe a correção

Para instalar (dentro de `playwright-project`):

```
npx playwright init-agents --loop=vscode
```

O comando cria as definições dos agentes e um *seed test*. No chat do VS Code, selecione o agente (ex.: **planner**) e peça: "*Crie um plano de testes para o carrinho da Hat Store*".

> Os agentes geram **rascunhos**. O plano precisa da sua revisão de QA (faltou cenário negativo?) e o código passa pelo mesmo checklist.

6.10 Checklist de revisão de código gerado por IA

Antes de aceitar um teste gerado ou editado pela IA:

- [ ] Os locators **existem** na página (confira no DevTools, no `codegen` ou no UI mode)?
- [ ] Não há `waitForTimeout` nem `sleep`?
- [ ] O teste roda **sozinho** e em qualquer ordem (massa própria)?
- [ ] As asserções verificam **o valor esperado** (`toHaveText('Cupom aplicado!')`), e não só "algo apareceu"?
- [ ] O teste **falha quando deveria**? Mude o valor esperado de propósito e veja ficar vermelho.
- [ ] Nenhum `test.skip`, mock ou asserção afrouxada foi colocado "para passar"?
- [ ] Usa os padrões do projeto (fixtures, Page Objects, `data/` e `factories/`)?
- [ ] Nome do teste em linguagem de negócio?

Segurança e IA

- **Nunca** cole segredos (senhas reais, tokens, `.env`) num prompt.
- **Nunca** cole código, dados de clientes ou documentos de projetos reais numa IA de **conta pessoal ou gratuita**. Siga a política da empresa.
- Dados de teste são sempre **fictícios** (use as factories).
- Se a IA sugerir desligar validações ("ignora o HTTPS", "pula a verificação do token"), trate como **alerta**. O teste existe para validar o comportamento real.

6.11 Se você tiver acesso ao código-fonte

> **Dica**
>
> **Bônus, não pré-requisito.** Tudo acima funciona sem o código. Se no seu projeto você tiver acesso:
>
> - dá para **confirmar regras** (ex.: o percentual real do cupom) direto na fonte, em vez de depender só da documentação;
> - dá para anexar o componente da tela como **contexto extra** para a IA;
> - mas **não** cole código proprietário numa IA de conta pessoal ou gratuita. Com a IA corporativa aprovada, siga as regras da empresa.

Exercício com IA

Bloco **"Bloco 6"** em exercicios.spec.js. Faça os dois testes do cupom **com a IA**:

1. Levante os fatos da tela (DevTools ou MCP).
2. Escreva o prompt no formato **contexto** → **fatos** → **tarefa** → **restrições**.
3. Gere, rode e passe pelo checklist.

4. **Anote** quantas mensagens você gastou e o que precisou corrigir. Vamos discutir no módulo 07.

Próximo módulo: 07 — Fechamento

Módulo 07 — Fechamento e próximos passos

7.1 Resolução do exercício de IA

Compare o seu teste do cupom com solucoes/exercicios.spec.js (bloco "Bloco 6") e responda:

- Quantas mensagens você gastou com a IA? O que mudaria no primeiro prompt para acertar de primeira?
- A IA inventou algum locator? Colocou algum `waitForTimeout`?
- A asserção dela verificava o **valor** (R$ 24.00 de desconto, total R$ 96.00) ou só "algo apareceu"?
- Você fez o teste falhar de propósito para garantir que ele pega o erro?

7.2 O que você construiu

- Uma suíte com **testes de tela, E2E, visuais, de API e de contrato**, organizada com Page Objects, fixtures, massa fixa e factory.
- **Um bug real encontrado e documentado** (`test.fail` do cupom HATOFF) e uma divergência de documentação (25% x 20%).
- Um método para usar IA **gratuita** sem depender do código-fonte: fatos da tela → prompt → revisão com checklist → MCP.

Próximos passos (para continuar estudando)

Tema	Por onde começar
CI/CD	Rodar a suíte no GitHub Actions a cada push: https://playwright.dev/docs/ci-intro
TypeScript	Migrar o projeto para TS (tipos ajudam a IA a errar menos): https://playwright.dev/docs/test-typescript
Autenticação reaproveitada	`storageState` para logar uma vez por execução: https://playwright.dev/docs/auth
Mock de rede	`page.route` para simular erros da API: https://playwright.dev/docs/mock
Acessibilidade	`@axe-core/playwright`: https://playwright.dev/docs/accessibility-testing
Visual no CI	Gerar baselines dentro de um container para não depender do SO
Test Agents	Aprofundar planner, generator e healer: https://playwright.dev/docs/test-agents

Documentação oficial: https://playwright.dev/docs/intro

Voltar ao início do curso